University of **P**aris-**E**st **C**réteil (UPEC)
École **P**ublique d'**I**ngénieurs de la **S**anté **E**t du **N**umérique (EPISEN)
Saint-Gobain GDII

How can we detect cheating effectively without compromising system security through kernel-level access?

Presented by Gilles MEUNIER

gilles.meunier@etu.u-pec.fr

May 2026

Jury:

Olivier MICHEL	Tutor
Hervé PERRACHON	Mentor
James ORTIZ	Teacher
Surname NAME	Teacher

Acknowledgements

I thank my friends for introducing me to Counter-Strike.

Contents

List of Figures iii

List of Tables iv

Introduction 1

0.1 Context 1

0.2 Problematic 2

0.3 Outline 2

1 State of the Art - Cheats and Anti-Cheats 3

1.1 Cheat-Based Exploration of System Architecture 3

1.1.1 Automation Cheats 3

1.1.2 Information Cheats 5

1.1.3 Advanced Cheats 6

1.2 Anti-Cheat Mechanisms 9

1.2.1 Client-Side Anti-Cheats 9

1.2.2 Kernel-Level Anti-Cheats 9

1.2.3 Data-Driven Detection Methods 9

2 Assessing Risks and Limitations of Kernel-Level Anti-Cheats using Windows-Kernel-Explorer 10

2.1 Structural Limitations of Kernel-Level Anti-Cheats 10

2.1.1 Technological Arms Race 10

2.1.2 Limits Against New Forms of Cheating 11

2.1.3 Economic and Operational Costs 11

2.2 Experimental Section: Illustrating Potential Intrusiveness 11

2.2.1 Experimental Objective 11

2.2.2 Experimental Observations 11

2.2.3 Critical Analysis 11

3 Server-Side Approaches: Development of a Gameplay Analytics Pipeline for Suspicion Scoring . . . 12
3.1 Pipeline Implementation . . . 13
3.1.1 Ingestion and Scoring Objectives . . . 13
3.1.2 Algorithmic Architecture and Core Concepts . . . 13
3.1.3 Practical System Deployment . . . 13
3.2 Dataset and Features . . . 13
3.2.1 Telemetry Data Model . . . 13
3.2.2 Data Sources and Profiling . . . 13
3.2.3 Data Constraints and Limitations . . . 14
3.3 Benchmark Execution and Raw Outputs . . . 14
3.3.1 Experimental Protocol . . . 14
3.3.2 Raw Evaluation Results . . . 14
3.4 Analysis and Performance Discussion . . . 14
3.4.1 Net Detection Results . . . 14
3.4.2 Mitigating the High-Skill Bias . . . 14
3.4.3 Future Optimizations . . . 14
3.5 Comparison with Kernel-level approach . . . 15
Conclusion . . . 16
Bibliography . . . 17
Glossary . . . 18
Acronyms . . . 21
A Socio-Legal, Privacy, and Community Dynamics of Kernel-Level Anti-Cheat Implementations . . . 22
A.1 System Visibility Level and Perceived Intrusion . . . 22
A.2 Social Perception and Community Acceptance . . . 23
A.3 Legal and Regulatory Questions . . . 24
B Other appendixes . . . 25
B.1 Cheaters in CS2 . . . 26
B.2 Existing data aggregation tool . . . 27

List of Figures

1 Comparison of the recoil from the AK-47 in-game with its theoretical model 4
2 Demonstration of wallhack using built-in command "r_aoproxy_show 1" 5
3 Ad from a cheating software provider showing wallhack in action 6
4 Privilege rings in an OS' structure 7
5 Infrastructure of a DMA cheat setup 8
6 User Segmentation & Acceptance Divide 23
7 Data aggregation made by a CS2 player to show the chance of a cheater being in a game 26
8 Existing data aggregation tool to help visualize a player's game statistics 27

List of Tables

Introduction

If you look at a multi-player game, it's the people who are playing the game who are often more valuable than all of the animations and models and game logic that's associated with it.

– Gabe NEWELL

0.1 Context

Online competitive video games, particularly first-person shooters (FPS), now occupy a central place in both the video game industry and the esports ecosystem. Their model relies on technical fairness between players: each participant must be subject to the same rules, mechanical constraints, and informational limitations. However, cheating represents a growing threat to this balance. The emergence of automated tools such as aimbots and wallhacks alters match integrity, undermines competitive credibility, and directly affects player trust as well as the economic viability of affected titles.

To address this issue, many publishers have progressively adopted kernel-level anti-cheat systems. These solutions operate with very high privileges, allowing them to monitor memory, processes, and certain hardware interactions within the operating system in order to detect manipulation of the game client. While often effective against certain forms of software cheating, they also raise significant concerns regarding cybersecurity, privacy protection, and user acceptability. The permanent presence of a third-party privileged component increases the system's attack surface, introduces a critical point of failure and fuels public debate about the proportionality between technical intrusion and its intended objective.

Beyond pure cybersecurity vulnerabilities, kernel-level solutions introduce profound privacy and regulatory dilemmas. Operating at the highest privilege layer (Ring 0) theoretically grants these software components unhindered visibility over the user's entire system, including personal files, background applications, and non-game-related data. This continuous monitoring mechanism challenges traditional notions of informed user consent and clashes directly with modern data protection frameworks, such as the GDPR, which mandate strict principles of data minimization and proportionality. Consequently, this technological choice has sparked a sharp divide within the gaming community: while competitive players often tolerate intrusive measures in the name of competitive integrity, casual users increasingly reject what they perceive as a form of digital surveillance, ultimately threatening game adoption and brand reputation.

0.2 Problematic

In this context, the following question arises: can cheating be effectively detected without compromising system security through kernel-level access? This thesis explores an alternative approach primarily based on server-side behavioral analysis and machine learning techniques. Instead of directly inspecting the player's machine, the idea is to leverage gameplay data itself—match statistics, action sequences, and temporal and mechanical consistency—to identify anomalies characteristic of assisted behavior.

Recent research suggests that analyzing replays and gameplay data in games such as Counter-Strike 2 makes it possible to distinguish legitimate players from suspicious profiles using labeled datasets. However, these approaches also present limitations: the need for large volumes of data, the risk of false positives, delayed detection, and the continuous adaptation of cheaters. The objective of this thesis is therefore twofold. First, to analyze the advantages and limitations of kernel-level anti-cheats in light of cybersecurity, system stability, and user trust considerations. Second, to design and evaluate a behavioral detection system based on gameplay metrics. A proof of concept will be implemented using replay data or simulated logs for a Counter-Strike-like FPS in order to generate a suspicion score and study the prioritization of human review.

Finally, the thesis will propose an architectural positioning of these approaches within a multi-layer anti-cheat system, combining server-side detection, lightweight client mechanisms, and human validation, with the aim of reconciling effectiveness, security, and player acceptability.

0.3 Outline

In the first chapter, we will present the state of the art in anti-cheat technologies, with a particular focus on kernel-level solutions and their implications. We will analyze their technical functioning, their effectiveness against various types of cheating, and the associated cybersecurity and privacy risks. In the second chapter, we will explore alternative approaches based on server-side behavioral analysis. We will review existing research on machine learning techniques applied to gameplay data and discuss their potential and limitations in the context of competitive FPS games. In the third chapter, we will present the design and implementation of a proof of concept for a behavioral detection system. We will describe the data collection process, the features extracted from gameplay metrics, and the machine learning models used to generate suspicion scores. We will also evaluate the system's performance and discuss the challenges of false positives and adaptive cheating. Finally, in the conclusion, we will synthesize our findings, discuss the implications for the future of anti-cheat technologies, and propose a multi-layered approach that combines server-side detection, lightweight client mechanisms, and human validation to effectively combat cheating while preserving system security and player trust.

Chapter 1

State of the Art - Cheats and Anti-Cheats

Technology is just a tool. In terms of getting the kids working together and motivating them, the teacher is the most important.

– Bill GATES

1.1 Cheat-Based Exploration of System Architecture 3
1.1.1 Automation Cheats 3
1.1.2 Information Cheats 5
1.1.3 Advanced Cheats 6
1.2 Anti-Cheat Mechanisms 9
1.2.1 Client-Side Anti-Cheats 9
1.2.2 Kernel-Level Anti-Cheats 9
1.2.3 Data-Driven Detection Methods 9

1.1 Cheat-Based Exploration of System Architecture

1.1.1 Automation Cheats

Automation cheats represent a critical class of malicious software engineered to augment or entirely substitute a player's mechanical execution within the competitive environment. This category primarily encompasses automated aiming tools (aimbots), automatic firing utilities (triggerbots), and execution macros or scripts. By targeting the core pillars of first-person shooter (FPS) gameplay—specifically mechanical precision, muscle memory, and cognitive reaction time, these tools disrupt competitive balance by replacing human motor control with software-driven operations.

To achieve this automation, the software must interface directly with either the operating system's input layer or manipulate the inner variables governing the game engine logic. The depth of this interaction generally defines the cheat's sophistication and detection difficulty:

- Input Layer Manipulation: Client-side automation tools often operate by injecting synthetic mouse and keyboard events directly into the operating system's input stream. A typical triggerbot, for example, continuously polls the game client's state or memory addresses to determine when an enemy entity's hitbox intersects with the exact coordinates of the player's crosshair. The moment this condition is validated, the cheat programmatically fires a click event into the input queue, completely bypassing human visual processing and nervous system latency. The latency between the detection of the event and the actual input can be configured to try and pass as a human reaction, though the main point of this process is still to be way faster than the average player.

- Subversion of Game Engine Logic: More advanced automation mechanisms bypass peripheral input emulation altogether, executing modifications directly within the local game engine client memory space. An aimbot reads the real-time spatial positions (three-dimensional coordinates) of all active opponents from the engine's local entity list. It then calculates the precise angular vector offsets required to bridge the gap between the current viewpoint and the target's head hitbox. By overwriting the client's view-angle variables within the engine memory before the next frame rendering or tick packet serialization, the cheat forces perfect accuracy.

Figure 1: Comparison of the recoil from the AK-47 in-game with its theoretical model

[1]

This enables frame-perfect recoil compensation and automated target tracking that completely overrides the physical constraints designed by the game developers.

[1] https://cs2pulse.com/spray-patterns/ak-47/

1.1.2 Information Cheats

Unlike automation tools that directly override a player's physical inputs, information cheats focus on subverting the tactical integrity of tactical shooter ecosystems by breaking down spatial boundaries. Tactical first-person shooters heavily rely on a conceptual "fog of war"—a structural design where information regarding enemy positions, utility placement, and movement is deliberately occluded by spatial geometry to reward map control and sensory awareness. Information cheats, primarily manifested as wallhacks and radarhacks, systematically dismantle this baseline restriction. They do not execute actions for the user but instead grant an absolute cognitive advantage by exposing hidden game data and altering the rendering pipeline.

To achieve this asymmetric visualization, these tools typically exploit two distinct technical vectors within the local client system:

- Rendering Pipeline Exploitation: Wallhacks operate by intercepting or altering the graphics rendering pipeline responsible for drawing the game world onto the player's screen. In a legitimate game loop, the engine utilizes occlusion to ensure that player models hidden behind opaque geometry (such as walls or solid terrain) are not drawn on the screen. Wallhacks circumvent this by hooking into the graphic APIs used by the engine. By injecting code that forces the graphics card to ignore occlusion parameters when processing enemy entity models, the cheat renders these entities on top of intervening geometry. This is frequently accompanied by ESP (Extra Sensory Perception) boxes, which overlay real-time lines, health bars, and item descriptions directly onto the player's view canvas.

Figure 2: Demonstration of wallhack using built-in command "r_aoproxy_show 1"

Figure 3: Ad from a cheating software provider showing wallhack in action

[2]

- Game Data Exposure: Radarhacks bypass the visual canvas entirely, targeting the underlying spatial data structures residing within the client's volatile memory or network socket streams. Every online game client maintains a local "Entity List" containing the precise three-dimensional coordinate vectors of all active participants, transmitted by the server to ensure local interpolation and hit registration. A radarhack continuously scans the process memory space to extract these spatial vectors or sniffs incoming network packets before they are parsed by the client. This data is then projected onto a secondary web-based interface, an external execution window, or a simulated 2D mini-map overlay. Because this methodology leaves the primary visual interface completely clean and does not alter rendering hooks, it provides the cheater with absolute situational awareness while significantly minimizing visibility to automated anti-cheat screen capture mechanisms or human review systems. This type of cheating is one of the hardest to detect as, except for the direct scanning of the process memory space, the behaviour of such cheaters can often be associated with a good game-sense or instincts, hence being harder to prove as blatant cheating.

1.1.3 Advanced Cheats

As client-side anti-cheat mechanisms evolved to actively monitor user-mode operations, the cheating ecosystem shifted toward highly sophisticated execution paradigms categorized as advanced cheats. These advanced threats deliberately bypass the operating system's standard application boundaries by executing within highly privileged environments or utilizing standalone external hardware vectors. By exploiting low-level OS/Kernel interactions and dedicated hardware interfaces, these tools minimize or

[2] https://anyx.gg/

entirely eliminate their local software footprint, rendering conventional user-mode memory scanners and API hooks ineffective.

- Kernel-Level Cheats and Kernel interaction: Kernel-level cheats operate inside the most privileged layer of the computer's processor architecture, known as Kernel Mode (Ring 0). While regular video games and applications are restricted to User Mode (Ring 3) where they have limited permissions, software running in Ring 0 has absolute control over the entire system's memory and hardware. Because modern client-side anti-cheat systems also run at this kernel level to monitor the game, cheat developers must find ways to gain equal authority to hide their presence and subvert security controls.

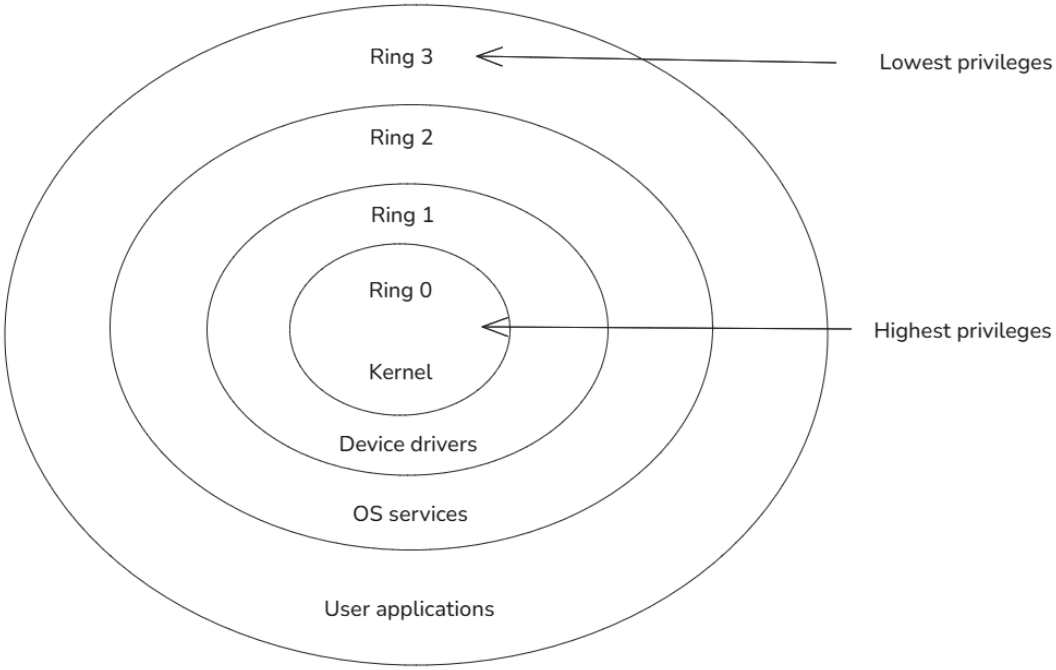

Figure 4: Privilege rings in an OS' structure

[3]

This advanced interaction with the operating system occurs through three primary mechanisms:

  - System Penetration via BYOVD: Modern operating systems like Windows require all kernel-level software to be digitally signed and approved by Microsoft. To bypass this security check, cheat developers use a technique called Bring Your Own Vulnerable Driver (BYOVD). They take an old, legitimately signed utility driver (from a trusted company) that contains a known security flaw. The cheat loads this trusted driver into the system and exploits its vulnerability to inject unsigned, malicious cheat code directly into the secure kernel memory space.
  - Direct System Manipulation (DKOM): Once established inside Ring 0, the cheat can modify the operating system's internal tracking structures. Using a method known as Direct Kernel

[3] https://www.tutorialspoint.com/article/protection-ring

Object Manipulation (DKOM), the cheat can erase its own program from the active process list or alter the system's memory maps (page tables). This disguises the cheat's memory footprint, making it look like a harmless, native operating system component.

- External Cheats and Hardware Interfaces: The most modern escalation in this technical conflict involves external and hardware-assisted cheats, which decouple the cheat execution environment from the host operating system running the game client. This paradigm relies on physical hardware interfaces, most notably Direct Memory Access (DMA) via specialized hardware expansion cards installed directly into the host machine's PCI Express (PCIe) slots. The DMA card physically reads and writes to the system's volatile memory (RAM) directly through the hardware controller, completely bypassing the CPU and any software-level security controls enforced by the operating system kernel. This raw memory stream is transmitted via a physical interface to a secondary, completely isolated computer. This external machine parses the game state asynchronously,calculates visual overlays or aim vectors, and can send synthetic peripheral inputs back to the host via specialized hardware input spoofers or modified mouse firmware. Because no malicious code executes, modifies memory, or hooks processes on the primary gaming machine, hardware-assisted vectors leave zero software footprint for traditional detection engines to scan.

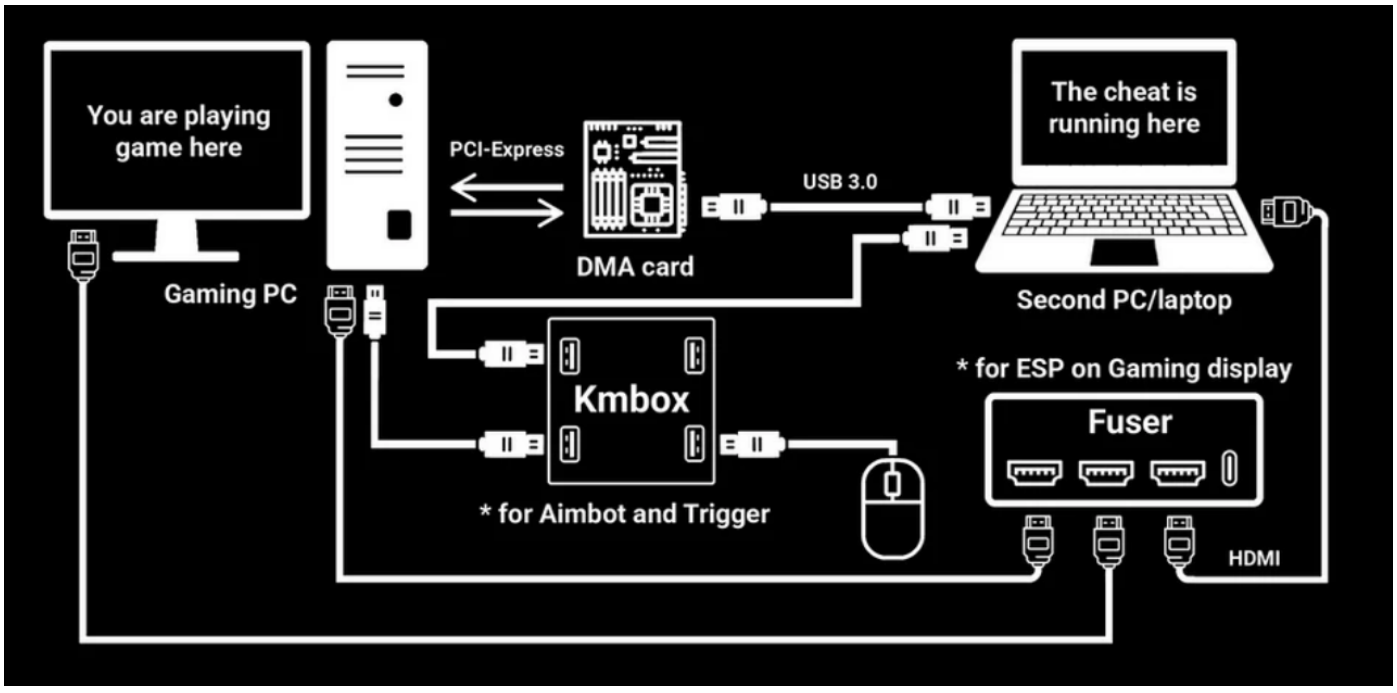

Figure 5: Infrastructure of a DMA cheat setup

4

[4] https://steams360.com/blogs/news/what-are-dma-cheats-and-how-does-it-work

1.2 Anti-Cheat Mechanisms

1.2.1 Client-Side Anti-Cheats

- Code injection detection
- Memory scanning

1.2.2 Kernel-Level Anti-Cheats

- Privileged monitoring
- Detection of low-level cheats

1.2.3 Data-Driven Detection Methods

- **Shifting the trust anchor to the server**
  - Reduced dependence on the client
  - Global view of game statistics
- **Player behavior modeling**
  - Performance patterns
  - Cross-match consistency
  - Statistical anomalies
- **Conceptual limitations**
  - False positives
  - Skilled player vs cheater bias
  - Exotic but legitimate behaviors

Chapter 2

Assessing Risks and Limitations of Kernel-Level Anti-Cheats using Windows-Kernel-Explorer

The programmers of tomorrow are the wizards of the future. You're going to look like you have magic powers compared to everybody else.

– Gabe NEWELL

2.1 Structural Limitations of Kernel-Level Anti-Cheats 10
2.1.1 Technological Arms Race 10
2.1.2 Limits Against New Forms of Cheating 11
2.1.3 Economic and Operational Costs 11
2.2 Experimental Section: Illustrating Potential Intrusiveness . . . 11
2.2.1 Experimental Objective 11
2.2.2 Experimental Observations 11
2.2.3 Critical Analysis 11

2.1 Structural Limitations of Kernel-Level Anti-Cheats

2.1.1 Technological Arms Race

- Constant adaptation of cheaters
- Increasing development costs
- Dependency on continuous updates

2.1.2 Limits Against New Forms of Cheating

- Cheat externalization outside the machine
- Hardware-assisted cheating

2.1.3 Economic and Operational Costs

- Driver maintenance and technical support
- Security incident management
- Reputational impact in case of issues

2.2 Experimental Section: Illustrating Potential Intrusiveness

2.2.1 Experimental Objective

- Illustrate the level of access of a privileged component
- Demonstrate security and privacy implications
- Headstart thanks to https://github.com/AxtMueller/Windows-Kernel-Explorer

2.2.2 Experimental Observations

- Access to sensitive system resources
- Privilege differences compared to regular software
- Observation of non-game-related data

2.2.3 Critical Analysis

- Real vs theoretical implications
- Link with user perception
- Discussion of effectiveness vs intrusiveness proportionality

Chapter 3

Server-Side Approaches: Development of a Gameplay Analytics Pipeline for Suspicion Scoring

We can't solve today's problems with the mentality that created them.

– Albert Einstein

3.1 Pipeline Implementation 13
3.1.1 Ingestion and Scoring Objectives 13
3.1.2 Algorithmic Architecture and Core Concepts 13
3.1.3 Practical System Deployment 13
3.2 Dataset and Features 13
3.2.1 Telemetry Data Model 13
3.2.2 Data Sources and Profiling 13
3.2.3 Data Constraints and Limitations 14
3.3 Benchmark Execution and Raw Outputs 14
3.3.1 Experimental Protocol 14
3.3.2 Raw Evaluation Results 14
3.4 Analysis and Performance Discussion 14
3.4.1 Net Detection Results 14
3.4.2 Mitigating the High-Skill Bias 14
3.4.3 Future Optimizations 14
3.5 Comparison with Kernel-level approach 15

3.1 Pipeline Implementation

3.1.1 Ingestion and Scoring Objectives

- Automating telemetry-driven suspicion scoring
- Prioritization matrix for human review (Overwatch-style triage)

3.1.2 Algorithmic Architecture and Core Concepts

- Rule-Based Foundations: Establishing game engine limits, behavioral baselines, and mechanical consistency boundaries
- Hybrid Modeling Framework: Combining a classical deterministic heuristic algorithm (live run execution) with sequential Machine Learning concepts (Long Short-Term Memory - LSTM)
- Flag Weighting and Temporal Aggregation: Make individual mechanical anomalies accumulate into a player history score

3.1.3 Practical System Deployment

- Architecture of the analytics pipeline: https://github.com/selligre/thesis-cs2-ac-server (could use https://github.com/pnxenopoulos/awpy to help parsing data from demos)
- Event-driven design: Comparison of real-time telemetry parsing vs. asynchronous post-match processing

3.2 Dataset and Features

3.2.1 Telemetry Data Model

- Aim Vectors: Time-to-Damage (TTD), pre-aim delta, and spray pattern deviation
- Spatial and Positional Features: Cross-match positioning consistency, Opening Duels context, and Trading Efficiency windows
- Tactical and Utility Metrics: Grenade/HE efficiency (Average HE damage), Clutch performance variance, and overall Kill/Death (KD)
- Social/Contextual Context: Premade lobby (Party) dependencies and cross-match behavioral

3.2.2 Data Sources and Profiling

- Synthetic telemetry generation: Simulating legitimate player profiles (varying from casual to highly-skilled) vs. simulated cheat profiles (aimbots, triggerbots, wallhacks and scripts)

3.2.3 Data Constraints and Limitations

- Limits of server-side logs
- Tick-rate limitations

3.3 Benchmark Execution and Raw Outputs

3.3.1 Experimental Protocol

- Setup of the controlled benchmarking environment
- Execution sequence: Batch processing of simulated profiles through the classical algorithm and the LSTM layer

3.3.2 Raw Evaluation Results

- Raw statistical outputs, distribution of suspicion scores and first results

3.4 Analysis and Performance Discussion

3.4.1 Net Detection Results

- Link between extreme data profiles and cheaters
- Robustness assessment: Evaluating the accuracy when mixing in high-skill profiles

3.4.2 Mitigating the High-Skill Bias

- Analyzing the conceptual limits of false positives
- Differentiating anomalous expert playstyles (exotic but legitimate behaviors) from software-assisted mechanics
- Comparative Analysis: Contrasting our suspicion-scoring pipeline with commercial performance platforms like Leetify.gg (How metrics designed for coaching can be repurposed for anti-cheat signaling)

3.4.3 Future Optimizations

- Build accurate data profiles faster
- Take into account external factors (number of cheaters in their friends list, age of the account, number of hours played related to the skill-level shown… things that experimented players learn to recognize but that the software fails to implement)

3.5 Comparison with Kernel-level approach

- Effectiveness
- Intrusiveness
- Implications for the FPS / e-sport industry
- Player perception
- Human validation

Conclusion

> *The easiest way to stop piracy is not by putting antipiracy technology to work. It's by giving those people a service that's better than what they're receiving from the pirates.*
>
> – Gabe NEWELL

- Overall synthesis
- Limitations and future work
  - Lack of real data
  - Adaptation to further cheating methods

Ultimately, this thesis demonstrates that shifting toward server-side behavioral analysis resolves not only the critical security flaws of kernel-level access but also its severe socio-legal friction. By relocating the trust anchor from the user's local machine to the game server, developers can entirely bypass the ethical dilemmas associated with continuous Ring 0 surveillance. This paradigm shift aligns seamlessly with stringent global privacy regulations like the GDPR by eliminating the collection of non-game personal data. Furthermore, by replacing intrusive local monitoring with non-invasive gameplay analysis, this approach restores user trust and removes the barrier to adoption for privacy-conscious players, proving that safeguarding competitive fairness does not require compromising fundamental digital rights.

Bibliography

[1] Christoph Dorner and Lukas Daniel Klausner. "If it looks like a rootkit and deceives like a rootkit: A critical examination of kernel-level anti-cheat systems". In: *Proceedings of the 19th International Conference on Availability, Reliability and Security*. Vienna Austria: ACM, pp. 1–11, July 2024 (see page 28).

[2] Mille Mei Zhen Loo, Gert Lužkov, and Paolo Burelli. "AntiCheatPT: A transformer-based approach to cheat detection in competitive computer games". In: *2025 IEEE Conference on Games (CoG)*. Lisbon, Portugal: IEEE, pp. 1–4, Aug. 2025 (see page 28).

[3] Sam Collins et al. "Anti-cheat: Attacks and the effectiveness of client-side defences". In: *Proceedings of the 2024 Workshop on Research on offensive and defensive techniques in the context of Man At The End (MATE) attacks*. Salt Lake City UT USA: ACM, pp. 30–43, Nov. 2024 (see page 28).

[4] Jakubowski Michał, Małgorzata Ćwil, and Szatkowska Weronika. "Enhancing Fair Play in online gaming: The development and implementation of the no more cheats anti-cheat system". en. In: *Lecture Notes in Computer Science*. Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, pp. 69–80, 2025 (see page 28).

[5] Y Kuzhii and Y Furgala. "The anticheat development in the counter-strike main series". In: *Electron. Inf. Technol.*, vol. 26, no. 760, 2024 (see page 28).

Glossary

A

Aimbot An automation cheat that systematically manipulates the view-angle variables stored within client process memory, enforcing automated frame-perfect vector corrections and target tracking that completely overrides the physical limitations of manual peripheral inputs. . . . **Page :** 28

C

Cheat Unsanctioned software, hardware, or system modifications designed to provide an asymmetric, unfair competitive advantage to a player by subverting authoritative game engine mechanics, spatial information visibility, or user-input processing pipelines. . . . . **Page :** 28

Client The local software instance of a video game running on the end-user's operating system, responsible for rendering local graphical vectors, capturing hardware peripheral inputs, and transmitting state packets asynchronously to a centralized authoritative network infrastructure. **Page :** 28

Counter-Strike A premier tactical round-based first-person shooter series developed by Valve, focusing on asymmetric objective match-ups between Terrorist and Counter-Terrorist factions, serving as the core empirical research and data evaluation framework for this thesis. . . **Page :** 28

Cybersecurity The scientific discipline and operational practice of protecting digital systems, networks, programs, and endpoints from unauthorized exploitation, access, or subversion, encapsulating the architectural evaluation of privileged kernel drivers versus server-side privacy boundaries. . . . . **Page :** 28

D

Driver A privileged kernel-mode software subsystem designed to abstract communications between the host operating system and physical hardware components, frequently weaponized by cheat developers via vulnerable third-party modules or deployed by anti-cheat systems for deep host visibility. . . . . **Page :** 28

E

E-Sport Electronic Sports, a structured global ecosystem of professional competitive video gaming organized into high-stakes leagues, where competitive integrity, technical fairness, and robust anti-cheat validation are paramount to financial and structural stability. . . . . **Page :** 28

F

First-Person Shooter A highly competitive genre of action video games centered on weapon-based combat rendered from a direct first-person graphical perspective, placing critical reliance on rapid mechanical precision, situational awareness, and split-second cognitive responses. . **Page :** 28

K

Kernel The foundational, core layer of an operating system operating at the highest execution privilege level (Ring 0), possessing unrestricted access to physical hardware channels, system volatile memory allocation, and low-level processor instructions. . . . . **Page :** 28

L

Leetify Leetify.gg, an external third-party data analytics platform that parses match-demo data streams to compute complex gameplay performance analytics (such as aim tracking and trading position efficiency), serving as the core conceptual inspiration for the server-side behavioral pipeline developed in this work. . . . . **Page :** 28

M

Microsoft Microsoft Corporation, the developer of the Windows operating system, whose secure boot architectures, device driver digital signature mandates, and low-level API design define the modern technical interface boundaries for client-side security software. . . . . **Page :** 28

O

Overwatch A decentralized crowdsourced tribunal system historically implemented within Valve's ecosystem, allowing experienced, trusted human reviewers to audit anonymized game replay telemetry to validate or reject suspicious flags produced by anti-cheat systems. . . . **Page :** 28

R

Radarhack An information cheat that extracts real-time physical spatial coordinates (X, Y, Z vectors) of hidden entities from volatile client memory space or intercepted network socket streams, projecting opponent positions onto an external clean secondary canvas or mini-map interface. **Page :** 28

Rings The hierarchical privilege rings architecture implemented within modern central processing units (CPUs) to isolate executing software, ranging from the highly restricted User Mode (Ring 3) to the absolute system authority of Kernel Mode (Ring 0). . . . . **Page :** 28

S

Script An execution sequence consisting of pre-compiled instructions or high-frequency input macros designed to automate complex, frame-dependent input timings, bypassing manual mechanical execution constraints such as perfect jump-throw synchronization or rapid spatial hops. **Page :** 28

Server The central, authoritative computing node in a multiplayer network architecture that validates game state logic, enforces simulation rules, manages synchronization across all connected clients, and generates deterministic telemetry data streams. . . . . **Page :** 28

T

Tick The elementary discrete discrete time-step execution unit of a multiplayer game server loop, defining the temporal frequency (measured in Hertz) at which the server processes client inputs, re-evaluates physical variables, and serializes state packets. . . . . **Page :** 28

Triggerbot An automation cheat that continuously polls internal entity lists or crosshair alignment parameters, instantly injecting synthetic mouse-click click signals into the operating system input stream the exact microsecond an opponent's hit-box intersects with the player's crosshair coordinates. . . . . **Page :** 28

V

Valve Valve Corporation, the American video game developer and digital distribution publisher responsible for creating the Source and Source 2 game engines, the Steam ecosystem, and the authoritative competitive Counter-Strike series. . . . . **Page :** 28

W

Wallhack A category of information cheat that intercepts and alters client-side graphics rendering application programming interfaces (APIs) by disabling occlusion culling or depth-testing rules, granting an unauthorized real-time visual overlay of opponent models through solid opaque terrain geometry. . . . . **Page :** 28

Acronyms

API Application Programming Interface . . . . . **Page :** 28

BYOVD Bring Your Own Vulnerable Driver . . . . . **Page :** 28

CPU Central Processing Unit . . . . . **Page :** 28

CS Counter-Strike . . . . . **Page :** 28

CS2 Counter-Strike 2 . . . . . **Page :** 28

CS:GO Counter-Strike: Global Offensive . . . . . **Page :** 28

DKOM Direct Kernel Object Manipulation . . . . . **Page :** 28

DMA Direct Memory Access . . . . . **Page :** 28

ESP Extra Sensory Perception . . . . . **Page :** 28

GRPD General Data Protection Regulation . . . . . **Page :** 28

HE High Explosive . . . . . **Page :** 28

LSTM Long Short-Term Memory . . . . . **Page :** 28

OS Operating System . . . . . **Page :** 28

PCIe Peripheral Component Interconnect Express . . . . . **Page :** 28

RAM Random Access Memory . . . . . **Page :** 28

TTD Time To Damage . . . . . **Page :** 28

Appendix A

Socio-Legal, Privacy, and Community Dynamics of Kernel-Level Anti-Cheat Implementations

A.1 System Visibility Level and Perceived Intrusion

The architectural integration of anti-cheat software into the kernel layer shifts the boundaries of data access. Operating at Ring 0, a driver possesses the same execution privileges as the core operating system, effectively rendering the traditional memory protection boundaries of user-mode (Ring 3) obsolete.

- Theoretical Access to Non-Game-Related Data: Because kernel drivers have unrestricted access to the entire Virtual Address Space, they can theoretically read and monitor memory pages belonging to unrelated applications. This includes open web browsers containing active sessions, password managers, encryption keys, and local files. While publishers assert that their drivers only scan for cheat signatures, the technical capability to perform deep packet and memory inspection across the entire system remains a valid point of concern for privacy advocates.
- Continuous vs. Occasional Monitoring: A major shift occurred with the introduction of Riot Games' Vanguard (vgk.sys), which mandates an "always-on" approach. Unlike traditional anti-cheats that initialize alongside the game client and terminate upon closure, always-on kernel drivers boot during the system startup sequence. The industry justification is to prevent the pre-loading of cheats or hypervisors before the anti-cheat can initialize. However, this creates a state of continuous monitoring, leaving a privileged third-party component active even when the user is performing sensitive tasks unrelated to gaming.
- The Notion of Implicit Consent: End-User License Agreements (EULAs) operate on a "take-it-or-leave-it" basis. Users are faced with a asymmetric choice: surrender Ring 0 access to their personal computer or lose access to the digital service entirely. In academic literature, this undermines the principle of freely given and informed consent, as the economic and social pressure to maintain connection with peer groups in modern video games forces users to accept invasive telemetric

conditions.

A.2 Social Perception and Community Acceptance

The deployment of kernel-level anti-cheats has transformed a purely technical challenge into a highly polarized cultural debate within the global gaming community.

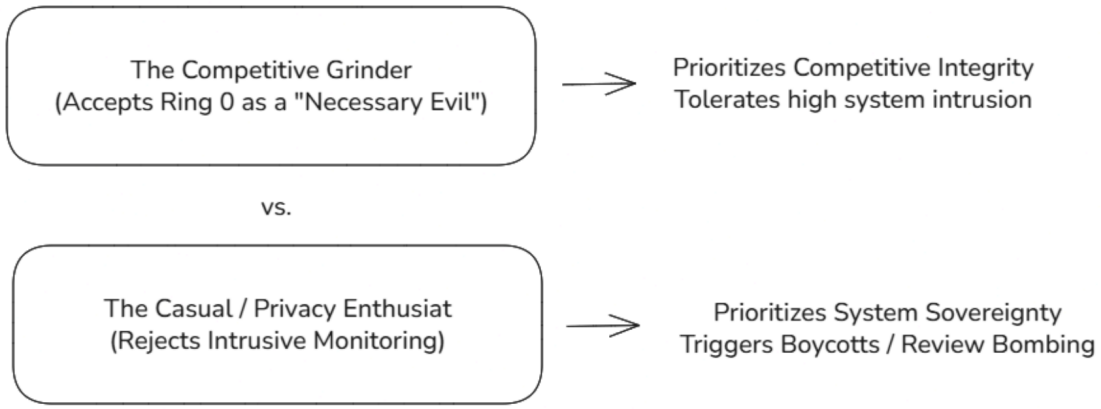

Figure 6: User Segmentation & Acceptance Divide

- Public Debates Around Intrusive Anti-Cheats: High-profile releases have frequently triggered severe community backlash. For example, the launch of Helldivers 2 faced significant criticism and review-bombing on platforms like Steam due to its implementation of nProtect GameGuard. Users frequently cite performance degradation, potential system instability, and the lack of transparency regarding what data is transmitted back to external servers as primary grievances.

- The Divide Between Competitive and Casual Players: The player base is structurally divided regarding acceptability. Highly competitive players, particularly within esports ecosystems, often champion intrusive client-side measures, viewing them as a necessary tool to safeguard competitive integrity and financial stakes. Conversely, casual players or hardware enthusiasts view these measures as disproportionate. This divide is further amplified by ecosystem fragmentation, such as the complete exclusion of Linux and Steam Deck users from titles like League of Legends or Apex Legends due to the technical incompatibility of kernel drivers with translation layers like Proton.

- Impact on Game Adoption: This friction directly impacts the commercial viability of software. A growing segment of privacy-conscious consumers actively boycotts games utilizing kernel-level solutions. For publishers, this creates an operational trade-off: mitigating financial losses caused by cheaters versus losing potential customers who refuse to compromise their system sovereignty.

A.3 Legal and Regulatory Questions

From a legal perspective, the deep monitoring required by client-side kernel components tests the boundaries of modern data protection frameworks and civil liability.

- GDPR Compliance: Proportionality and Data Minimization: Under Article 5(1)(c) of the General Data Protection Regulation (GDPR), personal data processing must be adequate, relevant, and limited to what is necessary in relation to the purposes for which they are processed (data minimization). Proving that an anti-cheat requires sweeping visibility over a user's entire operating system to catch cheaters represents a precarious legal position. If a server-side or behavioral approach can achieve similar outcomes without local system surveillance, the kernel-level approach may fail the core GDPR test of proportionality.
- Liability in Case of a Security Flaw: Standard EULAs explicitly disclaim all corporate liability for software-induced damages, hardware conflicts, or data breaches. However, from a consumer rights perspective, the mandatory installation of a software component capable of creating an exploitable system vulnerability (e.g., the historical Capcom driver exploit or the Genshin Impact anti-cheat exploit used to deploy ransomware) challenges these liability waivers. Should a major anti-cheat driver be weaponized on a global scale, publishers could face unprecedented class-action lawsuits concerning negligence in secure software design.
- Transparency and Terms of Service: Most publishers offer limited visibility into the telemetry data collected by their kernel components. The lack of open-source auditing or independent third-party verification means users must blindly trust corporate assertions. To align with modern transparency mandates, anti-cheat developers face growing pressure to provide detailed documentation, open APIs, or clear, granular logs showing precisely what memory segments are being analyzed and when data is being exfiltrated to the cloud.

Appendix B

Other appendixes

B.1 Cheaters in CS2

Figure 7: Data aggregation made by a CS2 player to show the chance of a cheater being in a game

1

[1] https://steamcommunity.com/app/730/discussions/0/4753074843849422337/

B.2 Existing data aggregation tool

Figure 8: Existing data aggregation tool to help visualize a player's game statistics

2

[2] https://csst.at/profile/76561198261637028

Abstract: Awesome abstract. [1, 2, 3, 4, 5] Cheat Client Server Kernel Valve Wallhack Radarhack Aimbot Triggerbot Script First-Person Shooter Counter-Strike E-Sport Cybersecurity Rings Microsoft Driver Overwatch Tick Leetify CS CS2 CS:GO OS DMA GRPD API ESP BYOVD DKOM PCIe RAM CPU LSTM TTD HE

Keywords: Key, Words.